Q1

A smart city has deployed an AI-powered ambulance navigation system. The road network is represented as a weighted graph where each edge contains both distance and traffic delay. The system has access to a heuristic function that estimates the straight-line distance from any intersection to the hospital. Initially, developers implemented Greedy Best-First Search to minimize estimated remaining distance, but during peak traffic hours, ambulances were getting delayed due to ignoring actual accumulated travel time. The system was later upgraded to use A* search to consider both distance traveled so far and estimated distance remaining.

Write a Python program to represent the road network as an adjacency list and implement both Greedy Best-First Search and A* search. Your implementation must compute and display the path taken, total travel time, and number of nodes expanded. Additionally, integrate a Simple Reflex Agent that immediately avoids any road whose traffic delay exceeds a given threshold, and a Goal-Based Agent that replans the entire route if a road becomes blocked during execution. Compare the outputs of both algorithms and analyze which performs better under heavy traffic conditions.

Q2

A delivery drone operates in a city modeled as a weighted graph where each edge represents energy consumption required to travel between two points. The drone has a limited battery capacity and must minimize energy usage while successfully reaching its destination. The heuristic function estimates straight-line distance to the delivery point.

Initially, the drone uses Greedy Best-First Search, but over time it begins to record actual energy consumption on frequently used routes and updates its heuristic estimates accordingly.

Design and implement Greedy Best-First Search and A* search for this system. The drone must operate as a Learning Agent that improves its heuristic estimates based on past experience (e.g., adjusting underestimated energy costs). After multiple simulation runs, compare how the learned heuristic affects path optimality and efficiency.

Your program should:

- Simulate multiple deliveries
- Update heuristic values based on observed costs
- Display path cost before and after learning
- Compare results with standard A* search

Q3

During a severe flood emergency, a city deploys an automated evacuation routing system to guide civilians from high-risk zones to designated safe areas. The city is represented as a weighted graph in which nodes correspond to locations and edges correspond to roads with associated travel time costs. During execution, certain roads may suddenly become flooded and unavailable without prior notice. You are required to implement both Greedy Best-First Search and A* search in Python to compute evacuation routes from a given start location to a safe zone using a suitable heuristic function such as straight-line distance. The system must operate strictly as a Simple Reflex Agent, meaning it can only react to the current percept (for example, detecting that a road is flooded) and must immediately remove flooded roads from consideration without storing previous states, learning from past experience, or performing complex replanning strategies beyond its immediate reaction. Your program should simulate dynamic road blockages during execution, display the final evacuation path found by each algorithm, compute total evacuation time, count the number of nodes expanded, and compare whether Greedy Best-First Search or A* search performs more reliably under sudden environmental changes when limited to purely reflex-based behavior.